Hollow Soul - M140 Manual Steps And Next Milestones

Build-real performance gate follow-up plan

This document turns the M140 implementation into an operator checklist and then proposes the next group of milestones. The immediate priority is to produce build-real evidence from macOS Apple silicon and Windows player artifacts before adding more optimization code.

Current State

- M140 gate code exists for player-side command-line capture, screenshot checks, player-log checks, render/shader reporting, and M138/M139 reuse.

- The macOS Apple silicon path can be run locally from this machine.

- The Windows runtime path still needs a Windows host or imported artifact folder. A macOS build alone cannot prove Windows runtime performance.

- Unity compile/import completed without C# compiler errors after M140 was added. The focused test runner did not emit an XML artifact in batch mode, so treat test execution as still pending.

Next Manual Steps

Step	Action	Pass Evidence
1	Open Unity, let it import, and confirm Console has zero compile errors.	No red compiler errors. M140 menus appear under Hollow/Performance.
2	Generate or refresh M140 assets from Hollow/Generation/Generate Milestone 140 Assets.	Assets/_Hollow/Data/Performance/M140BuildRealGateProfile.asset exists and lists macOS Apple silicon plus Windows x64.
3	Run the focused M140 EditMode tests from Test Runner, or batch them once Unity writes XML reliably.	Milestone140BuildRealGateTests passes. This proves parser, report, visual, log, import, and profile logic.
4	Run Hollow/Performance/Run M140 macOS Apple Silicon Gate.	output/reports/m140/m140_build_real_gate_editor.md plus macOS development and release-smoke player reports.
5	Inspect macOS screenshots and player logs from output/reports/m140/macos-apple-silicon/.	No dim overlay, missing camera render, hot-pink material, shader error, Addressables error, or runtime exception.
6	Build Windows artifacts from Unity if Windows Build Support is installed. Otherwise build on a Windows machine from the same commit.	Windows development and release-smoke players are produced from the same source revision.
7	On Windows, run the development player capture with the command below, then run release smoke.	A passing m140_build_real_gate.json and screenshots are produced on Windows.
8	Back on the main workstation, import Windows artifacts via Hollow/Performance/Import M140 Windows Player Artifacts.	M140 editor report changes from BlockedByEnvironment to Passed, unless a deterministic gate fails.

Step	Action	Pass Evidence
9	Only accept M140 when macOS Apple silicon and Windows runtime artifacts both pass.	M140 result is Passed. If blocked, the release gate is not complete yet.

Windows Capture Commands

Use paths that match the Windows machine. Keep the output folder intact when importing artifacts.

```
HollowSoul.exe --hollow-m140-capture --hollow-m140-auto-exit
--hollow-m140-output="C:\HollowSoul\m140\development" --hollow-m140-platform=windows-x64
--hollow-m140-build-kind=development --hollow-m140-scenarios=all --hollow-m140-fps-cap=60
```

```
HollowSoul.exe --hollow-m140-capture --hollow-m140-auto-exit
--hollow-m140-output="C:\HollowSoul\m140\release-smoke" --hollow-m140-platform=windows-x64
--hollow-m140-build-kind=release-smoke --hollow-m140-release-smoke --hollow-m140-scenarios=all
--hollow-m140-fps-cap=60
```

How To Triage M140 Results

Finding	Meaning	Next Fix
BlockedByEnvironment	The code path is ready but the required platform was not run.	Run on Windows or import Windows player artifacts. Do not mark release-ready.
p95 above 16.7 ms	The player build misses the 60 FPS comfort target.	Use scenario report to identify combat, traversal, branch load, or visual budget pressure.
Frame max above 50 ms	One-off hitch remains in loading, traversal, boss, or first-use path.	Check stage timings, cold cache misses, shader/material first-use, and pool misses.
Cold-cache misses after branch load	Branch loading did not preload something traversal needs.	Add the missed descriptor/presentation/pool/NavMesh item to branch load or boss load.
Runtime NavMesh fallback	Approved runtime content is missing baked/catalog NavMesh data.	Fix catalog bake/attach path. Do not cache runtime fallback as a normal solution.
Dim screenshot or curtain frames	A loading/transition UI is visible over playable state.	Fix reveal timing and overlay cleanup before optimizing anything else.
Shader/material log issue	A built player missed a shader/material/content path that Editor did not catch.	Add curated variants, fix material references, or adjust Addressables grouping.
M138/M139 failure	Combat scaling or long-run cache/pool behavior regressed in player.	Treat the original milestone report as the source of truth and repair there first.

Manual Visual Review Checklist

- Boot screen shows CineFit Studio / Hollow Soul cleanly, then fully clears before menu or gameplay.

- Branch entry loading hides work, but normal traversal has no dim or black overlay after reveal.

- Player silhouette, enemies, boss, projectiles, rewards, HUD, minimap, and room silhouettes read clearly in every screenshot.

- No first-use pop when entering reward, wave, projectile-heavy, or boss content.

- macOS Apple silicon report shows ARM/Apple silicon runtime evidence. Windows report shows Windows runtime evidence.

Next Group Of Milestones

These milestones should be run in order. The first two convert M140 from implemented tooling into trustworthy evidence. The later milestones harden the player-build pipeline so future optimization work cannot regress silently.

Milestone	Focus	Pass Gate
M141: M140 Evidence And Fix Pass	Run macOS Apple silicon and Windows M140 gates, then fix any deterministic failure before adding more systems.	Both platform runtime reports pass or failures are converted into explicit bug tasks with owner, scenario, and artifact link.
M142: Built-Player Scenario Fidelity	Make branch/traversal/reward/boss scenarios in the player harness drive real gameplay routes with stronger assertions.	Every M140 scenario proves it reached the intended gameplay state, not only a generic smoke capture.
M143: Release Build Performance Delta	Compare development telemetry builds against non-development release players and quantify overhead.	Release smoke is clean and release frame behavior is no worse than development evidence after accounting for missing telemetry.
M144: Shader Variant And Visual Regression Baselines	Create curated screenshot baselines and expand shader variant collection coverage from real player logs.	No shader/material first-use pop in branch, reward, projectile-heavy, or boss scenarios on both platforms.
M145: Built-Player Long Soak And Memory/GPU Drift	Move M139-style soak from Editor/short smoke into actual macOS and Windows players.	Long-run player memory, graphics memory, cache hit rate, pool counts, and recurring GC remain bounded.
M146: Runner Fleet And Artifact Discipline	Make M140-M145 reproducible through a documented local/Windows runner flow or CI machine.	Every release candidate has matching macOS and Windows artifact folders, commit hash, reports, logs, and screenshots.

Recommended Immediate Sequence

- Run M141 first. It is evidence work, not new architecture.

- If M141 exposes scenario ambiguity, run M142 before optimizing further.

- If M141 passes cleanly on both platforms, run M144 next because shader and visual pops are the most likely remaining player-only regressions.

- Run M145 only after the short player gates are stable; otherwise soak failures will be noisy.

- Use M146 once the manual process becomes repetitive enough to deserve runner automation.

Milestone Details

M141: M140 Evidence And Fix Pass

- Run macOS Apple silicon development and release-smoke gates from the M140 menu.
- Run or import Windows development and release-smoke artifacts from a Windows host.
- Review every screenshot and player log, then fix failures in the source milestone: M138 for combat scale, M139 for cache/pool soak, M140 for build/runner/reporting.
- Produce a final M140 editor report with Passed status or a failure docket.

Pass: macOS Apple silicon and Windows runtime artifacts both pass from the same commit.

M142: Built-Player Scenario Fidelity

- Replace any remaining generic scenario smoke with route-specific player harness steps.
- For branch entry, traversal, return, reward, boss, and next branch, assert the exact room role and expected counters.
- Add per-scenario readiness predicates so screenshots are captured only after gameplay is visible and stable.
- Add report fields that state which branch room, role, enemy count, reward state, and route were reached.

Pass: M140 scenarios cannot pass unless the intended gameplay state was actually reached.

M143: Release Build Performance Delta

- Run paired development and release players for macOS and Windows.
- Compare frame p95/max directionally, screenshot state, player logs, shader issues, and startup/branch-load time.
- Document expected telemetry differences between development and release builds.
- Fail if release builds are visually wrong or show content/shader errors even when development builds pass.

Pass: release smoke is clean and any dev/release performance delta is understood.

M144: Shader Variant And Visual Regression Baselines

- Collect screenshots from every M140 scenario on both platforms.
- Promote approved screenshots to lightweight baselines for dimming, missing render, HUD/minimap visibility, and material sanity checks.
- Mine player logs for missing variants/material issues and update curated ShaderVariantCollection assets.
- Keep blanket Shader.WarmupAllShaders forbidden.

Pass: no shader/material first-use miss and no visual baseline regression in player builds.

M145: Built-Player Long Soak And Memory/GPU Drift

- Run multi-branch soak inside built macOS and Windows players.
- Track managed memory, graphics memory, cache entries, pool active/rented/inactive counts, GC p95, and shader/material misses after warmup.
- Include save/load, branch abandon/re-enter, boss room, reward room, next branch, and return traversal.
- Compare first branch warmup against later traversal windows.

Pass: memory remains bounded and no post-load cold misses or stale pooled state appear.

M146: Runner Fleet And Artifact Discipline

- Standardize folder layout for macOS and Windows reports, screenshots, logs, and build manifests.
- Add one scriptable command for local macOS gate and one for Windows host gate.
- Make import validation strict: platform id, build kind, commit hash, scenario coverage, and pass status must match.
- Document how a release candidate is accepted or rejected.

Pass: any engineer can reproduce the evidence package without manual guesswork.